崇实战队27赛季算法组第一次作业

概述

> 这次作业主要分为两个部分，分为文本部分和编程部分。 主要考察简单的代码编程能力，以及动手操作解决问题的能力，同时为了解这段时间的学习情况，需同学们自学并使用markdown语言将第一阶段学习内容及成果做成报告提交，此外，整个作业需打包提交至自己github的仓库，将自己的仓库地址填写在群（QQ群聊：469113615）里的收集表中（收集表预计在作业发布后24h内发出，请及时关注后续群消息）。作业截止时间为10.15（周四）23：30前。

ps.收集表与作业完成时间截止时间一致，请完成任务的同学一定记得在群里填写自己的仓库地址

学习进度汇报

> markdown是一种轻量级标记语言，排版语法简洁，让人们更多地关注内容本身而非排版。它使用易读易写的纯文本格式编写文档，可与HTML混编，可导出 HTML、PDF 以及本身的 .md 格式的文件。因简洁、高效、易读、易写，Markdown被大量使用，如Github、Wikipedia、简书等。

千万不要被「标记」、「语言」吓到，Markdown的语法十分简单，常用的标记符号不超过十个，用于日常写作记录绰绰有余，不到半小时就能完全掌握。
此处附上几个参考文件和网站，大家可以快速学习markdown语法，也可以边整理学习笔记，边翻看资料(^^)

- 官方资料：markdown官方教学文档
- b站上这个up主的视频讲解的非常好，大家也可以跟着视频进行学习40分钟教会你用免费的vscode做出好看的笔记

经过一段时间的学习，相信大家已经学习了不少知识。当然，在学习知识的同时肯定也遇到过不少问题，这里建议大家在学习的时候可以通过学习笔记或者markdown的形式将这些问题和找到的学习方法记录下来，以避免后续遇到相同的问题不会解决。因此，第一部分的任务是需要大家使用Markdown语言完成第一阶段学习进程的汇报，并且完成后将文档转为pdf格式，对于转格式方面有不懂的可以在网上查询，这里不再赘述。文件结构包含.md和.pdf两项。 在文件夹中展示2个简单例子，供各位参考，详情见**附件1和附件2**。

编程题

数组反转

题目描述

> 💻先来个简单题热热身

```
// 原地反转数组：将 arr 中的元素顺序颠倒
// arr 是数组指针，n 是数组长度
void reverseArray(int* arr, int n);
```

输入：

1. 第一行一个整数 n（1 <= n <= 100）
2. 第二行n个整数

输出：反转后的数组，每个元素之间用空格隔开
例子：

```
输入：
5
1 2 3 4 5

输出：
5 4 3 2 1
```

要求：

1. 必须原地修改数组，不能新建数组
2. 必须用指针遍历数组
3. 有必要的注释
4. 处理 n=0 的情况

爬楼梯

题目描述：

> 你正在爬一个 n 阶的楼梯，每次可以爬 1 阶或 2 阶。求爬到第 n 阶共有多少种不同的方法。

接口要求：

```
// 计算爬到第 n 阶楼梯的方法数（用迭代实现）
long long climbStairsIter(int n);

// 计算爬到第 n 阶楼梯的方法数（用递归实现）
long long climbStairsRecur(int n);
```

输入：一个整数 n（0 <= n <= 40）
输出：两种方式的计算结果 + 对比两种方法是否结果一致
要求：

1. 处理 n = 0 和 n = 1 的边界情况
2. 分别用迭代和递归实现，并在 main 里对比结果
3. 部分必要的注释
4. 可以尝试n取更大值的时候两种方法的区别👀

🤔什么时候适合用递归，什么时候适合用迭代呢🤔

二叉树中的遍历

题目描述：

> 二叉树本质上是一个“每个节点最多有两个分支”的数组结构。例如：

```
假设有一个数组tree = {1, 2, 3, 4, 5, -1, -1}，并且规定：
1. 下标 0 的元素 1 是“根”
2. 下标 1 的元素 2 是 1 的“左孩子”
3. 下标 2 的元素 3 是 1 的“右孩子”
4. 下标 3 的元素 4 是 2 的“左孩子”
5. 下标 4 的元素 5 是 2 的“右孩子”
6. -1 表示“这个位置是空的”
那么这棵树长这样：
        1
       / \
      2   3
     / \
    4   5
```

> 现在你不需要定义结构体，只需要用数组 + 下标 + 递归，就能完成三种遍历。对于节点下标 i，左孩子下标为 2*i+1，右孩子下标为 2*i+2。选择你理解的两种遍历方式即可，三种都会做当然更好，但不必强求。

接口要求：

```
// 前序遍历：根 -> 左 -> 右
void preorderTraversal(const int* tree, int n, int i);

// 中序遍历：左 -> 根 -> 右
void inorderTraversal(const int* tree, int n, int i);

// 后序遍历：左 -> 右 -> 根
void postorderTraversal(const int* tree, int n, int i);

// 构建一棵简单的测试树（填充数组）
void buildTestTree(int* tree, int n);
```

输入：无
输出：你所选择的遍历方法所遍历出来的结果
要求：

1. 用数组 tree[ ] 存储树，用下标 i 表示当前节点
2. 递归时，左孩子下标 2*i+1，右孩子下标 2*i+2
3. 必须处理越界情况：如果 i >= n 或 tree[i] == -1（表示空节点），直接返回
4. 必要的注释

提示（举例）：

1. 有效节点有 5 个，但数组需要覆盖到最深的空节点位置。
2. 根节点下标是 0，节点 3 的下标是 2，它的孩子位置下标是 5 和 6。
3. 所以数组长度 n 至少要取到 7，才能把“空节点”标记出来。

🤔算一下，buildTestTree 里 n 应该传多少（n 只要“足够大到覆盖所有可能被访问的下标”即可，可以取 7，也可以取更大，自己判断）。

成绩统计与安全查询系统设计

题目描述：

> 请你完成一个成绩统计与安全查询系统的设计，要求包含一下功能：
>
> 1. 初始化成绩数组
> 2. 录入成绩
> 3. 计算平均分
> 4. 安全查询
> 5. 统计分数段
> 6. 找最高分与最低分

接口要求：

```
// 1. 初始化成绩数组（n为学生人数，scores为学生分数）
void initScores(int* scores, int n);

// 2. 录入成绩
void inputScores(int* scores, int n);

// 3. 计算平均分
double calcAverage(const int* scores, int n);

// 4. 安全查询
// 目的：确保任何查询都不会导致程序崩溃
// 返回值：如果下标越界，打印“下标越界”并返回 -1；如果下标合法但成绩未录入，也返回 -1。
int safeGet(const int* scores, int n, int index);

// 5. 统计分数段（counts为各个分数段的个数）
void countGrades(const int* scores, int n, int* counts);

// 6. 找最高分与最低分
void findMinMax(const int* scores, int n, int* max, int* min);

//7.打印所有结果
void printResults(const int* scores, int n, double avg, const int* counts, int max, int min);

int main() {
    int scores[MAX_STUDENTS];
    int n;
    int counts[5] = {0};
    int max = -1, min = -1;
    double avg = 0.0;
    int testIndex;

    // 1. 初始化数组
    initScores(scores, MAX_STUDENTS);

    // 2. 读取学生人数
    std::cout << "请输入学生人数：";
    std::cin >> n;

    // 3. 调用各函数（要求 0 <= score <= 100）
    inputScores(scores, n);

    avg = calcAverage(scores, n);

    findMinMax(scores, n, &max, &min);

    countGrades(scores, n, counts);

    // 7. 安全查询测试
    std::cout << "请输入要查询的下标：";
    std::cin >> testIndex;
    int queried = safeGet(scores, n, testIndex);
    if (queried == -1) {
        std::cout << "查询失败" << std::endl;
    } else {
        std::cout << "下标 " << testIndex << " 的成绩是：" << queried << std::endl;
    }

    // 8. 打印所有统计结果
    printResults(scores, n, avg, counts, max, min);

    return 0;
}
```

除了功能完整之外，希望大家可以注意到边界安全性和工程规范性，多考虑一些异常情况，让自己的代码更加健壮(✧◡✧)

作业提交

将两部分作业完成后打包在一起，并提交至github（若因其他问题导致无法进入，可提交至gitee），随后将自己所在作业的仓库地址填写至群里收集表中，收集表截止时间与作业时间一致，逾期不候。 文件结构包含：

1. 作业原题文件
2. 编程部分文件
3. 学习进度文件（包含.md和.pdf两种形式）
4. 其他

为了让初学者更快了解git和github，在此依旧提供一些简单教程供大家参考，鼓励大家自己搜寻更好的资源和学习教程，可分享至群中共同进步。

- 来自广州航海学院ICEBREAKER破冰船机器人实验室的培训材料，花费1.5小时就能掌握VScode+git进行代码管理，内容简明概要，非常适合初学者学习。vscode+git本地代码版本管理及远程仓库保存
- 通过短短20min讲明git的工作流及基本原理，并且具有常用的github基本操作，通过形象的比喻讲清git的工作流程，易于理解，讲解的也很清晰。git基本工作流+github基本操作

结语

第一次作业相对来说比较基础，希望大家能投入一定的时间和精力，认真对待，在干中学，相信一次作业的收获会远超你前期的学习，不以完成作业为目的，量力而行，同时做到精益求精，将自己前期积累的知识转化为解决实际问题的能力，有任何问题可以直接在群中询问或者私信管理员。最后，在国庆节假期的尾巴祝大家国庆节快乐～

> 参与编者：罗婉童
> 通讯信息：QQ 2826223614